LESSON 09 · CHEAT SHEET

Terminal Operations & Reduction

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. One terminal op runs the pipeline. Pick it by the **kind of answer**: elements, one value, yes/no, one element. 8
2. `toList()` is **unmodifiable** and allows nulls. `Collectors.toList()` makes no promise. 16
3. `forEachOrdered` keeps encounter order in parallel; `forEach` does not.
4. **reduce**: identity → T · no identity → Optional · 3-arg → U (with combiner).
5. The operator must be **associative** and the identity **neutral**, or parallel breaks.
6. Money: `reduce(ZERO, BigDecimal::add)`, then `divide(n, 2, HALF_UP)`. Guard empty.
7. Empty stream: **allMatch / noneMatch = true**, anyMatch = false.

Terminal operations

Operation	Returns	Note	Since
`forEach / forEachOrdered`	void	side effects	8
`collect(Collector)`	R	L10, L11	8
`toList()`	List (unmod.)	nulls ok	16
`toArray(String[]::new)`	String[]	typed	8
`reduce(id, op)`	T	empty → id	8
`reduce(op)`	Optional<T>	empty → empty	8
`reduce(id, acc, comb)`	U	type change	8
`count()`	long		8
`min / max(cmp)`	Optional<T>	add a tie-breaker	8
`any/all/noneMatch`	boolean	✂ short-circuit	8
`findFirst / findAny`	Optional<T>	✂ short-circuit	8

Canonical snippets

1 · Total balance (money)

```
accounts.stream().map(Account::balance)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
// 300382.25
```

2 · Average premium, rounded

```
total.divide(BigDecimal.valueOf(n),
             2, RoundingMode.HALF_UP)
// 10826.50 / 10 = 1082.65
```

3 · Highest, any, first

```
txns.stream()
    .max(comparing(Transaction::amount))
// Optional[T049 7721.99]
accounts.stream()
    .anyMatch(a -> a.balance().signum() < 0)
// true (ACC-1013)
```

The two reduce rules (real output)

```
reduce(0, (a, b) -> a - b)   seq -55, parallel ≠
reduce(100, Integer::sum)    seq 155, parallel >
reduce(ZERO, BigDecimal::add) 300382.25 always ✓
```

Empty stream results

anyMatch	allMatch	noneMatch	reduce(id,op)	max / findFirst
false	true	true	identity	Optional.empty

Real results from the course code

Practice	Result
total balance	300382.25
highest transaction	T049 7721.99
any overdrawn account	true (ACC-1013)
average premium	1082.65

Call	Result
`toList().add(..)`	UnsupportedOperationException
`toUnmodifiableList()` + null	NullPointerException
USD at 1.3456, round per line	1947.75 (at end: 1947.74)
Oct 2026 debits (exercise)	5832.65 SGD

Anti-patterns → fixes

Don't	Do
`reduce(OPENING, add)`	`reduce(ZERO, add).add(OPENING)`
`divide(n)`	`divide(n, 2, HALF_UP)` + empty guard
`forEach(list::add)`	`toList()` / `collect`

Don't	Do
`allMatch` as validation	check `isEmpty()` too
`toList().add(..)`	`toCollection(ArrayList::new)`
`reduce(0, sum)` for ints	`mapToInt(..).sum()`

Lesson 09 · sticky notes

Cut out, stick on your wall, and read one every morning.

toList() is read-only

Java 16 · add() throws.
Need to add? toCollection(ArrayList::new).

Money sum

`reduce(ZERO, BigDecimal::add)`
No sum() for BigDecimal.

divide needs a scale

`divide(n, 2, HALF_UP)`
and never divide by an empty count.

3 forms of reduce

id+op → T · op → Optional
id+acc+combiner → U

Associative + identity

a−b and "starting balances"
break in parallel.

Empty stream

allMatch = true · noneMatch = true
anyMatch = false

max returns Optional

Highest txn: T049 7721.99.
Add thenComparing(id) for ties.

findFirst vs findAny

First = encounter order.
Any = cheaper in parallel.